

Name - Aditya Pratap Singh Bhadoria  
Enroll no. - 24114005

## **Data Encryption Standard (DES) Report**

### **→ Some Historical context :-**

DES originated in the early 1970s from IBM's Lucifer design and was adapted and standardized by the U.S. government as FIPS PUB 46 (published 1977). DES became the U.S. federal data encryption standard and remained widely used for two decades. The official FIPS description and parameterization (block size, key size, permutations, S-boxes, and round schedule) were documented and reaffirmed through several revisions until being withdrawn in 2005 (while Triple DES was recommended as an interim measure).

### **→ Overview :-**

DES is a Feistel cipher, a structure that splits 64-bit plaintext block into two 32-bit parts (L,R) and iterates a round transformation 16 times. The Feistel construction has two benefits: (1) it makes decryption trivial (apply the same round function with round keys used in reverse order), and (2) it allows complex non-invertible round functions (the round function  $f$  need not be invertible). The standard parameters are:

- **Block size:** 64 bits.
- **Key:** 64 bits input but only 56 bits used for cryptographic keying (8 parity bits).
- **Rounds:** 16.

These parameters are part of the formal FIPS specification.

### **→ Detailed Working :-**

A block to be enciphered is subjected to an initial permutation **IP**, then to a complex key-dependent computation and finally to a permutation which is the inverse of the initial permutation **IP**<sup>-1</sup>. The key-dependent computation can be simply defined in terms of a function **f**, called the cipher function, and a function **KS**, called the key schedule. Next is the explanation of each part.

## 1. Enciphering

The 64 bits of the input block to be enciphered are first subjected to the initial permutation IP. The permuted block is then passed to a complex key-dependent computation. The output of that computation, called the preoutput, is then subjected to the inverse of the initial permutation,  $IP^{-1}$ .

| <u>IP</u> |    |    |    |    |    |    |   |
|-----------|----|----|----|----|----|----|---|
| 58        | 50 | 42 | 34 | 26 | 18 | 10 | 2 |
| 60        | 52 | 44 | 36 | 28 | 20 | 12 | 4 |
| 62        | 54 | 46 | 38 | 30 | 22 | 14 | 6 |
| 64        | 56 | 48 | 40 | 32 | 24 | 16 | 8 |
| 57        | 49 | 41 | 33 | 25 | 17 | 9  | 1 |
| 59        | 51 | 43 | 35 | 27 | 19 | 11 | 3 |
| 61        | 53 | 45 | 37 | 29 | 21 | 13 | 5 |
| 63        | 55 | 47 | 39 | 31 | 23 | 15 | 7 |

| <u>IP<sup>-1</sup></u> |   |    |    |    |    |    |    |
|------------------------|---|----|----|----|----|----|----|
| 40                     | 8 | 48 | 16 | 56 | 24 | 64 | 32 |
| 39                     | 7 | 47 | 15 | 55 | 23 | 63 | 31 |
| 38                     | 6 | 46 | 14 | 54 | 22 | 62 | 30 |
| 37                     | 5 | 45 | 13 | 53 | 21 | 61 | 29 |
| 36                     | 4 | 44 | 12 | 52 | 20 | 60 | 28 |
| 35                     | 3 | 43 | 11 | 51 | 19 | 59 | 27 |
| 34                     | 2 | 42 | 10 | 50 | 18 | 58 | 26 |
| 33                     | 1 | 41 | 9  | 49 | 17 | 57 | 25 |

Now let the input block (64 bits) be divided as L + R (each 32 bits), and denoted as LR, and K be a block of 48 bits chosen from the 64 bits key. Then rounds working is as follows :-

$$(1) L' = R \mid R' = L \oplus f(R, K) \quad (\oplus = \text{xor})$$

At each iteration a different block K of key bits is chosen from the 64-bit key designated by key.

$$(2) K_n = KS(n, \text{key})$$

Let KS be a function which takes an integer n in the range from 1 to 16 and a 64-bit block KEY as input and yields as output a 48-bit block  $K_n$  which is a permuted selection of bits from KEY.  $K_n$  is used in the nth iteration in (1).

$$(3) L_n = R_{n-1} \mid R_n = L_{n-1} \oplus f(R_{n-1}, K_n)$$

The nth iteration block is  $L_n R_n$ . However, the preoutput block is  $R_{16} L_{16}$ .

The key schedule KS produces the 16  $K_n$  which are required for the algorithm.

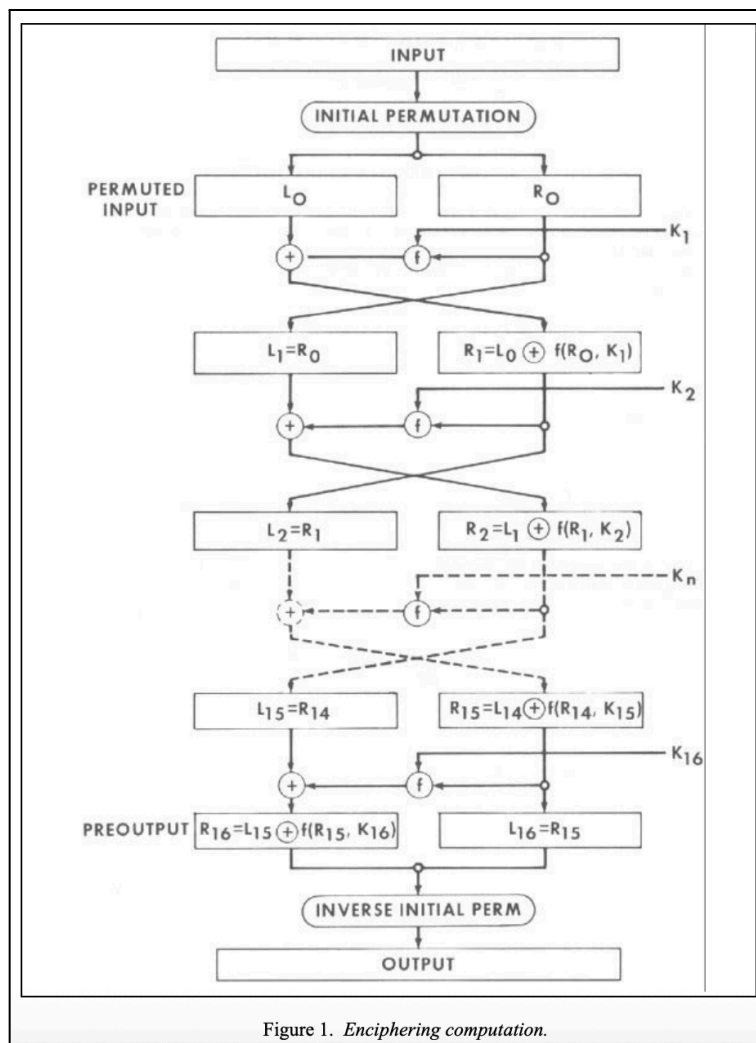


Figure 1. Enciphering computation.

## 2. Deciphering

The permutation  $IP^{-1}$  applied to the preoutput block is the inverse of the initial permutation  $IP$  applied to the input. Further, from above it follows that:

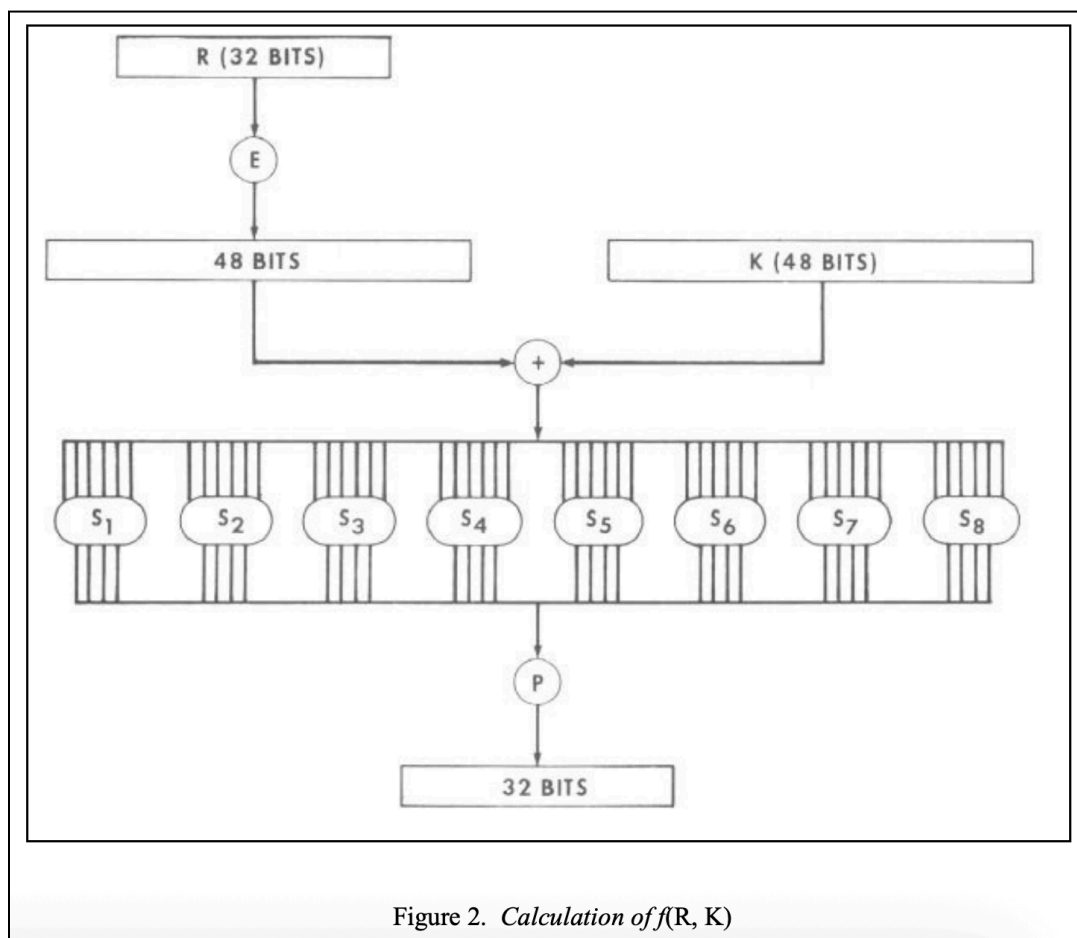
$$R = L' \quad | \quad L = R' \oplus f(L', K)$$

To decipher it is only necessary to apply the very same algorithm to an enciphered message block, taking care that at each iteration of the computation the same block of key bits  $K$  is used during decipherment as was used during the encipherment of the block. Using the notation of the previous section, this can be expressed by the equations:

$$R_{n-1} = L_n \quad | \quad L_{n-1} = R_n \oplus f(L_n, K_n)$$

Now  $R_{16}L_{16}$  is the permuted input block for the deciphering calculation and  $L_0R_0$  is the preoutput block.

## 3. The Cipher function $f$



Let **E** be a function which takes a block of 32 bits as input and yields a block of 48 bits as output. **E** be such that the 48 bits of its output, written as 8 blocks of 6 bits each, are obtained by selecting the bits in its inputs in order according to the following table:

| <u><b>E BIT-SELECTION TABLE</b></u> |    |    |    |    |    |
|-------------------------------------|----|----|----|----|----|
| 32                                  | 1  | 2  | 3  | 4  | 5  |
| 4                                   | 5  | 6  | 7  | 8  | 9  |
| 8                                   | 9  | 10 | 11 | 12 | 13 |
| 12                                  | 13 | 14 | 15 | 16 | 17 |
| 16                                  | 17 | 18 | 19 | 20 | 21 |
| 20                                  | 21 | 22 | 23 | 24 | 25 |
| 24                                  | 25 | 26 | 27 | 28 | 29 |
| 28                                  | 29 | 30 | 31 | 32 | 1  |

Each of the unique selection functions **S<sub>1</sub>, S<sub>2</sub>, ..., S<sub>8</sub>**, takes a 6-bit block as input and yields a 4-bit block as output as illustrated in the below table containing **S<sub>1</sub>**:

| Row<br>No. | Column Number |    |    |   |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|---------------|----|----|---|----|----|----|----|----|----|----|----|----|----|----|----|
|            | 0             | 1  | 2  | 3 | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
| 0          | 14            | 4  | 13 | 1 | 2  | 15 | 11 | 8  | 3  | 10 | 6  | 12 | 5  | 9  | 0  | 7  |
| 1          | 0             | 15 | 7  | 4 | 14 | 2  | 13 | 1  | 10 | 6  | 12 | 11 | 9  | 5  | 3  | 8  |
| 2          | 4             | 1  | 14 | 8 | 13 | 6  | 2  | 11 | 15 | 12 | 9  | 7  | 3  | 10 | 5  | 0  |
| 3          | 15            | 12 | 8  | 2 | 4  | 9  | 1  | 7  | 5  | 11 | 3  | 14 | 10 | 0  | 6  | 13 |

If **S<sub>1</sub>** is the function defined in this table and **B** is a block of 6 bits, then **S<sub>1</sub>(B)** is determined as follows: The first and last bits of **B** represent in base 2 a number in the range 0 to 3. Let that number be *i*. The middle 4 bits of **B** represent in base 2 a number in the range 0 to 15. Let that number be *j*. Look up in the table the number in the *i*'th row and *j*'th column. It is a number in the range 0 to 15 and is uniquely represented by a 4 bit block. That block is the output **S<sub>1</sub>(B)** of **S<sub>1</sub>** for the input **B**.

The permutation function  $P$  yields a 32-bit output from a 32-bit input by permuting the bits of the input block, as given in the following table:

| <u><math>P</math></u> |    |    |    |
|-----------------------|----|----|----|
| 16                    | 7  | 20 | 21 |
| 29                    | 12 | 28 | 17 |
| 1                     | 15 | 23 | 26 |
| 5                     | 18 | 31 | 10 |
| 2                     | 8  | 24 | 14 |
| 32                    | 27 | 3  | 9  |
| 19                    | 13 | 30 | 6  |
| 22                    | 11 | 4  | 25 |

Now define  $B_1, \dots, B_8$  to be blocks of 6 bits each for which

$$B_1 B_2 \dots B_8 = K \oplus E(R)$$

Then  $f(K, R)$  is defined as :

$$P(S_1(B_1)S_2(B_2)\dots S_8(B_8))$$

Each  $B_i$  is taken as an input to  $S_i$  and the 8 blocks  $S_1(B_1)S_2(B_2)\dots S_8(B_8)$  of 4 bits each are consolidated into a single block of 32 bits which forms the input to  $P$ .

The output is then the output of the function  $f$  for the inputs  $R$  and  $K$ .

#### 4. Keys

The round key,  $K_n$ , for  $1 \leq n \leq 16$ , is the block of 48 bits. To describe  $KS$ , it is sufficient to describe the calculation of  $K_n$  from  $KEY$  for  $n = 1, 2, \dots, 16$ . One bit in each 8-bit byte of the Key may be utilized for error detection in key generation, distribution and storage. Bits 8, 16, ..., 64 are for use in assuring that each byte is of odd parity.

First the 64-bit key is permuted using PC-1 to give two 28-bit blocks  $C_0$  and  $D_0$ . The bits of  $C_0$  are respectively bits 57, 49, 41, ..., 44 and 36 of  $KEY$ , with the bits of  $D_0$  being bits 63, 55, 47, ..., 12 and 4 of  $KEY$ .

Permuted choice 1 is determined by the following table:

**PC-1**

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| 57 | 49 | 41 | 33 | 25 | 17 | 9  |
| 1  | 58 | 50 | 42 | 34 | 26 | 18 |
| 10 | 2  | 59 | 51 | 43 | 35 | 27 |
| 19 | 11 | 3  | 60 | 52 | 44 | 36 |
| 63 | 55 | 47 | 39 | 31 | 23 | 15 |
| 7  | 62 | 54 | 46 | 38 | 30 | 22 |
| 14 | 6  | 61 | 53 | 45 | 37 | 29 |
| 21 | 13 | 5  | 28 | 20 | 12 | 4  |

Now,  $C_n$  and  $D_n$  are given by left shifting (actually left rotating) the bits of  $C_{n-1}$  and  $D_{n-1}$  by number of bits given in the below table:

| Iteration Number | Number of Left Shifts |
|------------------|-----------------------|
| 1                | 1                     |
| 2                | 1                     |
| 3                | 2                     |
| 4                | 2                     |
| 5                | 2                     |
| 6                | 2                     |
| 7                | 2                     |
| 8                | 2                     |
| 9                | 1                     |
| 10               | 2                     |
| 11               | 2                     |
| 12               | 2                     |
| 13               | 2                     |
| 14               | 2                     |
| 15               | 2                     |
| 16               | 1                     |

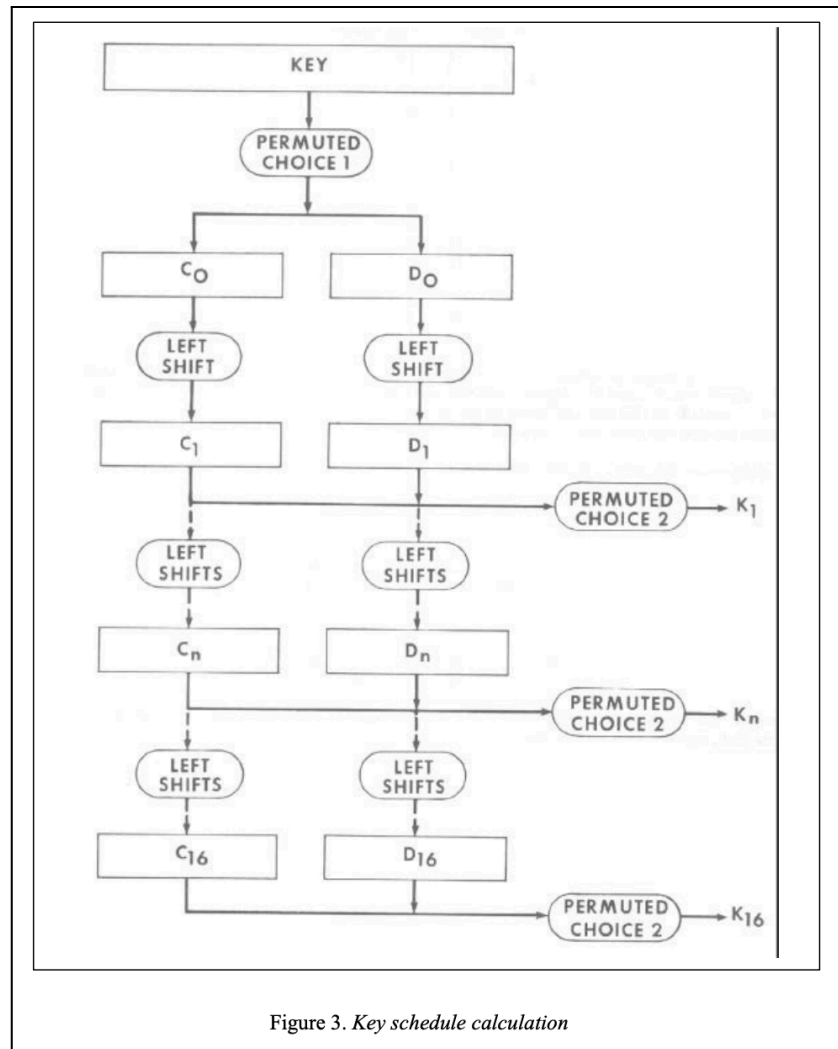


Figure 3. Key schedule calculation

Permuted choice 2 is determined by the following table:

| <u><b>PC-2</b></u> |    |    |    |    |    |
|--------------------|----|----|----|----|----|
| 14                 | 17 | 11 | 24 | 1  | 5  |
| 3                  | 28 | 15 | 6  | 21 | 10 |
| 23                 | 19 | 12 | 4  | 26 | 8  |
| 16                 | 7  | 27 | 20 | 13 | 2  |
| 41                 | 52 | 31 | 37 | 47 | 55 |
| 30                 | 40 | 51 | 45 | 33 | 48 |
| 44                 | 49 | 39 | 56 | 34 | 53 |
| 46                 | 42 | 50 | 36 | 29 | 32 |

So, the first bit of  $K_n$  is the 14th bit of  $C_n D_n$ , the second bit the 17th, and so on with the 47th bit the 29th, and the 48th bit the 32nd.